What the Arcezia gate does in the default state

One page, no jargon, for whoever reads the report second. Every number here is from the 30 decisions in the accompanying record.

It holds almost everything. With no contracts, checks or probes registered, the gate sent nearly every action to a person and blocked five outright. It allowed nothing without a person in the twenty-three rows reconciled against its own feed. A harmless read was held alongside a production drop. That is the correct direction for a default to fail in, and it is why nothing in the report is urgent.

It does not rank danger within a single action. A safe read, a `DROP TABLE` against production, and a mass delete with no WHERE clause all came back with the same four required items and the same summary sentence. The difference between them lives in the action's text, where only a human reading it can see it. In the default state this is a queue problem. In an instrumented account, where approvals start clearing, it becomes the difference between a reviewer seeing a routine read and an irreversible drop.

It is a holding gate, not an allow-or-block gate. The interesting question in this state is not what gets blocked. It is what turns a hold into a release. The short answer: scope, a person's approval, and for database actions a recent verified backup. Nothing the agent can produce by itself releases anything.

The sequence is the blind spot. A personal-data read followed by an outbound email, and a database export followed by a file write followed by an external email, both ran to completion. Neither was blocked, and neither produced any signal that joined the steps. Inside the chain, the first read was checked against fewer requirements than the same read sent alone. If a chain plan is what turns this on, then a new customer has no way to know their chains are unmonitored until they write one.

Three things it does better than expected. An agent cannot approve itself: malformed tokens are ignored, valid tokens without a registered signing key are held and explicitly not counted, and only a registered key clears an approval. An agent cannot shrink its own declared scope to sneak an action past the gate, because the declaration is checked against what the action actually does, and the lie is the violation. And the credentials file is walled against read, write and copy alike.

A privilege escalation is cheaper to release than the thing it authorises. An agent launch conferring the production-admin role was held against three items: scope, a rate-limit check and a user approval. A production database write is held against a production approval as well. So the action that hands out production authority clears one gate earlier than a single action performed under it. Chained behind a harmless launch, a production drop came back needing a person but was not blocked, and nothing in the response named the privilege change.

Two smaller things. A malformed approval token leaves the certificate's `signature` field empty, where every other decision returned a hex value; the decision is right, the certificate is not. And the Live Verify page's sample code does not match the shipped 1.0.8 SDK, which is a docs fix and is listed separately from the gate findings.

What this page does not cover. The instrumented account. Contracts, checks, a registered signing key and chain plans change the system enough that none of the above should be assumed to carry over. That is the phase two conversation, and the report is structured so it shows exactly which findings a phase two would close.